Week 14

ID1063 Introduction to Programming in C

Lecture 25: Strings, File I/O, Stream Processing, #define

String library functions

(string.h: working with '\0'-terminated text)

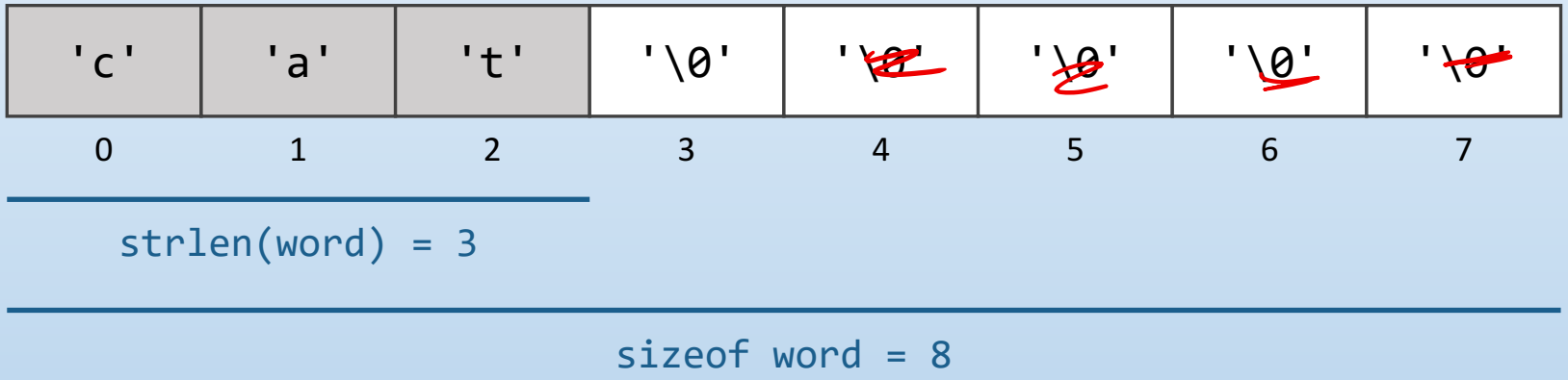
Strings and string.h

- A string is a `char` array that ends with the null character `'\0'`.
- `string.h` provides functions that work up to that `'\0'`:

Function	What it does
`strlen(s)`	length: characters before `'\0'`
`strcpy(dest, src)`	copy `src` into `dest`
`strcat(dest, src)`	append `src` to the end of `dest`
`strcmp(a, b)`	compare: 0 if equal, negative or positive otherwise
`strchr(s, c)`	find the first character `c` in `s`
`strstr(s, t)`	find the string `t` inside `s`

strlen: length

```
char word[8] = "cat";
printf("%zu %zu\n", strlen(word), sizeof(word));    /* 3 8 */
```



- `strlen` walks the array until it meets `'\0'`
- The `‘\0’` is **not counted**.
- %zu is for an unsigned integer, since the lengths are non-negative.

strcpy: copying a string

- An array cannot be assigned with =: copy the characters with `strcpy`.

```
Student st = {42, "Asha", 72.5};
st.name = "Divya";                  /* error: array */
strcpy(st.name, "Divya");           /* OK */
```

- Note: `strcpy` **does not check the size**: `dest` should have room for `strlen(src) + 1` characters.

```
if (strlen(src) + 1 <= sizeof(dest) )
    strcpy(dest, src);
```

strcat: joining strings

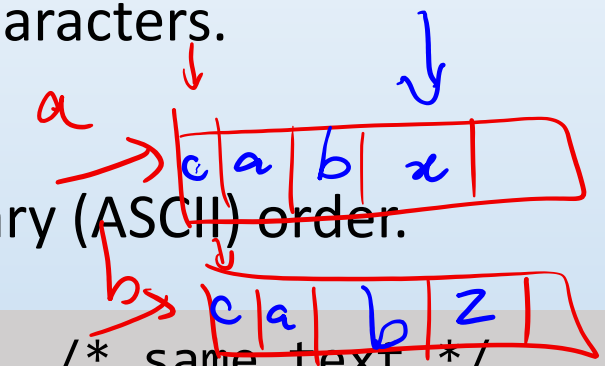
- `strcat(dest, src)` finds the `'\0'` of `dest` and copies `src` there.

```
char full[20] = "Asha";
strcat(full, " ");
strcat(full, "Rao");
printf("%s (%zu)\n", full, strlen(full));
/* prints: Asha Rao (8) */
```

- `dest` must have room for both parts plus `'\0'`:
  - `strlen(dest) + strlen(src) + 1 <= sizeof dest`

strcmp: comparing strings

- `a == b` compares two **addresses**, not the characters.
- `strcmp` compares character by character,
- Return value depends on ordering in dictionary (ASCII) order.

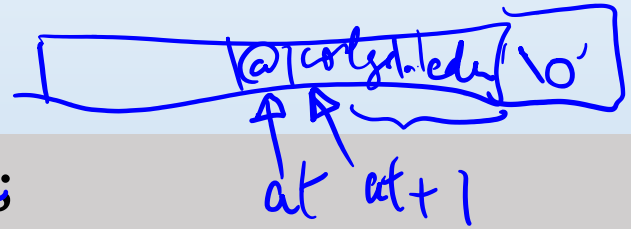



```
if (strcmp(cls[i].name, "Meera") == 0)      /* same text */
    printStudent(&cls[i]);
```

Call	Result
`strcmp("Asha", "Asha")`	0 (equal)
`strcmp("Asha", "Ravi")`	negative: "Asha" comes first
`strcmp("cat", "car")`	positive: 't' comes after 'r'

Searching: strchr and strstr

- Both return a **pointer** to where the match starts, or NULL if there is none.




```
char email[] = "asha@college.edu";
char *at = strchr(email, '@');
if (at == NULL)
    printf("not an email\n");
else
    printf("domain: %s\n", at + 1);     /* college.edu */

if (strstr(line, "Meera") != NULL)  /* found "Meera" */
    printf("%s", line);
```

pointer arithmetic

File input and output

(streams: the keyboard, the screen, and files)

Streams you already know

- A **stream** is a sequence of characters flowing into or out of a program.
- Three streams are open when every C program starts:

Stream	Connected to	Used by
`stdin`	keyboard	`scanf`, `getchar`
`stdout`	screen	`printf`, `putchar`
`stderr`	screen (for errors)	`perror`

- A file opened with `fopen` is just one more stream, of type `FILE *`.
- The same functions, with an `f` in front, work on any stream: `fprintf`, `fscanf`, `fgets`.

Opening and closing a file

```
FILE *fp = fopen("class.txt", "r"); /*reading mode*/
if (fp == NULL) {
    printf("Error opening class.txt");  /* prints the reason */
    return 1;
}
/* ... read from fp ... */
fclose(fp);
```

Mode	Meaning	If the file exists	If not
"r"	read	read from the start	fails: NULL
"w"	write	**erases** it first	creates it
"a"	append	adds at the end	creates it

FILE is internally a typedef'ed struct that is a file stream.
Contains state of read, current position in stream, etc.

Writing to a file: fprintf

- `fprintf` is `printf` with a stream as its first argument.

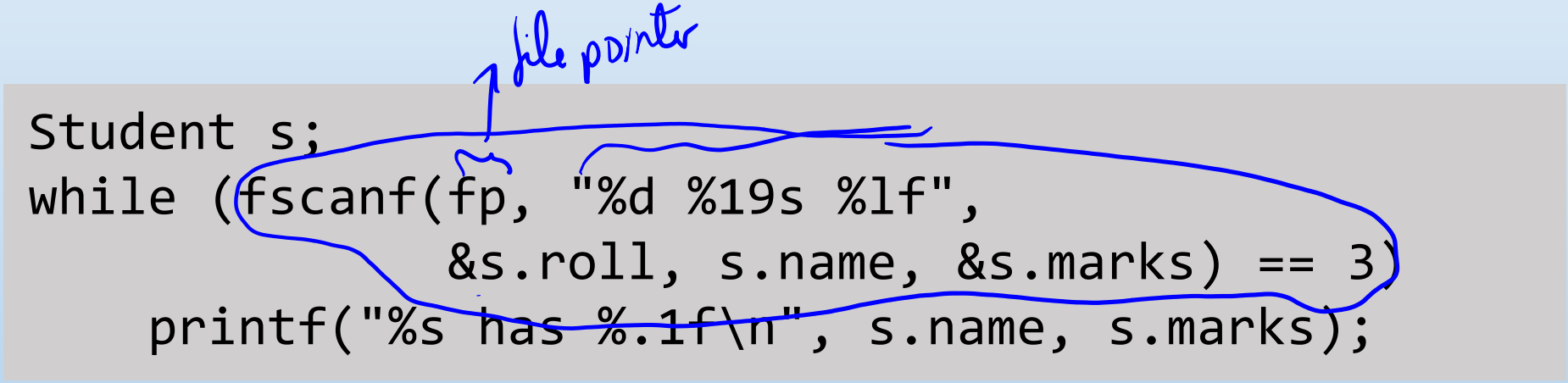
```
FILE *out = fopen("report.txt", "w");
if (out == NULL)
    return 1;

fprintf(out, "Roll Name Marks\n");
fprintf(out, "%d %s %.1f\n", s.roll, s.name, s.marks);
fclose(out);
```

- `printf(...)` is the same as `fprintf(stdout, ...)`.
- Open `report.txt` to see the output.

Reading with fscanf

- `fscanf` is `scanf` reading from a stream.
- It returns how many items it read: stops when format does not match



- At the end of the file it returns **`EOF (-1)`**
- On a badly formatted line, returns 0.

Reading a line: fgets

```
char *fgets(char *buf, int size, FILE *stream);
```

- Reads up to `size - 1` characters, and stops after a newline `'\n'`.
- Keeps the `'\n'`, adds `'\0'`; returns `NULL` at the end of the file.
- Never writes past `buf`: the size is part of the call.

```
char line[100];
while (fgets(line, sizeof line, fp) != NULL)
    printf("Read: %s", line);     /* line already ends in \n */
```

fgets works on stdin too

- `scanf("%s", name)` stops at the first space: "Asha Rao" becomes "Asha".
- `stdin` is a stream, so `fgets` reads a whole line from the keyboard.

```
char name[40];
printf("Full name: ");
if (fgets(name, sizeof name, stdin) != NULL) {
    name[strcspn(name, "\n")] = '\0';   /* drop '\n' */
    printf("Hello, %s!\n", name);
}
```

- `strcspn(name, "\n")` (in `string.h`) gives the position of the first `'\n'`.

Read a line, then parse it: sscanf

- `sscanf` is `scanf` reading from a string.
- **Recommended**: Read each line with `fgets`, then take it apart with `sscanf`.

```
char line[100];
Student s;
while (fgets(line, sizeof line, fp) != NULL) {
    if (sscanf(line, "%d %s %lf",
                &s.roll, s.name, &s.marks) == 3)
        printf("%s has %.1f\n", s.name, s.marks);
    else
        printf("Skipping bad line: %s", line);
}
```

- A bad line is reported and skipped; the rest of the file is still read.

Quick check: files

- `report.txt` already has 50 lines. What happens after `fopen("report.txt", "w")`?
- `fopen("data.txt", "r")`, but there is no `data.txt`. What does `fopen` return?
- `char buf[10]; fgets(buf, 10, fp);` reads at most how many characters?
- The input line is `Asha Rao`. What does `scanf("%s", name)` store? And `fgets`?